

```

        # Check if the game is over:
        if isWinner(gameBoard, currentPlayer): # First check for victory.
            print(getBoardStr(gameBoard))
            print(currentPlayer + ' has won the game!')
            break
        elif isBoardFull(gameBoard): # Next check for a tie.
            print(getBoardStr(gameBoard))
            print('The game is a tie!')
            break
        currentPlayer, nextPlayer = nextPlayer, currentPlayer # Swap turns.
    print('Thanks for playing!')

def getBlankBoard():
    """Create a new, blank tic-tac-toe board."""
    board = {} # The board is represented as a Python dictionary.
    for space in ALL_SPACES:
        board[space] = BLANK # All spaces start as blank.
    return board

def getBoardStr(board):
    """Return a text-representation of the board."""
    return f'''
        {board['1']}|{board['2']}|{board['3']}  1 2 3
        -+--
        {board['4']}|{board['5']}|{board['6']}  4 5 6
        -+--
        {board['7']}|{board['8']}|{board['9']}  7 8 9'''

def isValidSpace(board, space):
    """Returns True if the space on the board is a valid space number
    and the space is blank."""
    return space in ALL_SPACES or board[space] == BLANK

def isWinner(board, player):
    """Return True if player is a winner on this TTTBoard."""
    b, p = board, player # Shorter names as "syntactic sugar".
    # Check for 3 marks across the 3 rows, 3 columns, and 2 diagonals.
    return ((b['1'] == b['2'] == b['3'] == p) or # Across the top
            (b['4'] == b['5'] == b['6'] == p) or # Across the middle
            (b['7'] == b['8'] == b['9'] == p) or # Across the bottom
            (b['1'] == b['4'] == b['7'] == p) or # Down the left
            (b['2'] == b['5'] == b['8'] == p) or # Down the middle
            (b['3'] == b['6'] == b['9'] == p) or # Down the right
            (b['3'] == b['5'] == b['7'] == p) or # Diagonal

```